perplexity

Detailed Notes: OOP Principles (Inheritance, Polymorphism, Encapsulation, Abstraction) – Kunal Kushwaha

These notes follow the topic structure of Kunal Kushwaha's *OOP 3 | Principles - Inheritance, Polymorphism, Encapsulation, Abstraction* and incorporate key code examples and concepts to provide a comprehensive understanding for all levels.

1. Principles of OOP

Java OOP is built on four pillars:

- **Inheritance**
- **Polymorphism**
- **Encapsulation**
- **Abstraction**

Mnemonic: *IPEA* ("I Paid Eighty American").

2. Inheritance

Inheritance means acquiring properties and behaviors from one class (parent) into another (child). It models real-world relationships and supports code reuse.

Syntax:

```
class Parent {
    int x = 10;
}
class Child extends Parent {
    int y = 20;
}
```

Usage:

```
Child c = new Child();
System.out.println(c.x); // Inherited value from Parent
```

Types of Inheritance

- **Single Inheritance:** One child, one parent.
- **Multiple:** Java does not directly support. Achieved via interfaces.
- **Multi-level:** A derives from B, B derives from C. (Grandparent-parent-child)
- **Hierarchical:** Multiple classes inherit from a single parent.
- **Hybrid:** Combination (limited in Java; interfaces used).

Real-world Example: Box Class

```
class Box {
    double length, width, height;
    Box(double l, double w, double h) {
        length = l; width = w; height = h;
    }
}
class BoxWeight extends Box {
    double weight;
    BoxWeight(double l, double w, double h, double wt) {
        super(l, w, h); // Calls Box constructor
        weight = wt;
    }
}
```

super keyword

- Invokes parent constructor or methods.

```
super(argumentList);
super.methodName();
```

3. private Keyword and Encapsulation

- Members marked **private** aren't accessible outside their class. Enforces "data hiding".
- Applied to variables/methods except in special access scenarios.

```
class Example {
    private int value;
    public int getValue() { return value; }
}
```

Why private? Prevents unauthorized access and modification.

4. Polymorphism

Means “many forms”: the same entity can behave differently.

Types:

- **Compile-time (Static) Polymorphism:** Achieved via method overloading.
- **Run-time (Dynamic) Polymorphism:** Achieved via method overriding.

Method Overloading (Static Polymorphism)

Same method name, different parameters.

```
class MathOp {
    int add(int a, int b) { return a+b; }
    double add(double a, double b) { return a+b; }
}
```

Method Overriding (Dynamic Polymorphism)

Subclass changes parent’s implementation.

```
class Animal {
    void speak() { System.out.println("Animal"); }
}
class Dog extends Animal {
    void speak() { System.out.println("Dog"); }
}
```

Usage:

```
Animal obj = new Dog();
obj.speak(); // Output: Dog
```

Java uses **dynamic method dispatch**—the reference type’s object at runtime determines which method is called.

5. Overloading vs. Overriding

Overloading	Overriding
Same method name, different parameter	Same method name, same parameter
Within same class	Parent-child class relationship
Resolved at compile-time	Resolved at runtime

6. final Keyword

- Applied to variables, methods, classes.
  - **Variable:** Value can't change
  - **Method:** Can't override
  - **Class:** Can't extend

```
final class Constants { }
final int MAX = 100;
```

7. Can We Override Static Methods?

No. Static methods belong to the class, not an object. Declaring a static method with the same name in child hides parent's version, but doesn't override.

8. Encapsulation

Encapsulation = Data + Code bundled together + Hiding internal details.

How?

- Make fields private.
- Provide public getter/setter methods.

Example:

```
class Person {
    private String name;
    public String getName() { return name; }
    public void setName(String n) { name = n; }
}
```

9. Abstraction

Abstraction = Show only relevant details, hide complexity.

Example:

- Abstract classes or interfaces.
- User doesn't see how methods work internally, only method signatures.

10. Encapsulation vs Abstraction vs Data Hiding

Concept	Purpose	Example
Encapsulation	Bundle data/methods, restrict access	Private fields, getters/setters
Abstraction	Hide complex implementation, show essentials	Interfaces, abstract classes

Concept	Purpose	Example
Data Hiding	Restrict access to internals	Private/protected access

11. Types of Inheritance – Code Examples

Single:

```
class A {}
class B extends A {}
```

Multi-level:

```
class A {}
class B extends A {}
class C extends B {}
```

Hierarchical:

```
class A {}
class B extends A {}
class C extends A {}
```

Multiple & Hybrid: Use interfaces:

```
interface X {}
interface Y {}
class Z implements X, Y {}
```

12. Real-World Example—Shapes

```
class Shape {
    void draw() { System.out.println("Drawing Shape"); }
}
class Circle extends Shape {
    void draw() { System.out.println("Drawing Circle"); }
}
class Square extends Shape {
    void draw() { System.out.println("Drawing Square"); }
}

Shape s = new Circle();
s.draw(); // Output: Drawing Circle
```

13. Dynamic Method Dispatch Example

```
Shape shapeRef;
shapeRef = new Square();
shapeRef.draw(); // Output: Drawing Square
shapeRef = new Circle();
shapeRef.draw(); // Output: Drawing Circle
```

14. Data Hiding

Make member variables private and provide only public access methods as required. Prevents external code from tampering with internal state.

15. Summary Table

Principle	Description	Achieved via	Example
Inheritance	Acquire from parent class	extends / implements	`class B extends A {}`
Polymorphism	Many forms	Overloading, overriding	Same method, different classes
Encapsulation	Bundle data, hide details	private, getters/setters	`private int a;`
Abstraction	Show essentials only	abstract, interfaces	`abstract class A {}`

Best Practices & Key Points

- Prefer composition over inheritance if the relationship doesn't fit "is-a".
- Minimize deep inheritance hierarchies.
- Use `super` to call parent constructors/methods.
- Use access modifiers wisely: `private` protects, `public` exposes.
- Remember: static methods and fields are not inherited like instance methods.

These notes comprehensively cover the core concepts, detailed code, and critical points from the video—serving as an excellent revision and interview preparation resource[1] [2] [3].

⁂

1. https://www.youtube.com/playlist?list=PL9gnSGHSqcno1G3XjUbwzXHL8_EttOuKk
2. https://sansverse.co/3-intro-to-object-oriented-programming--4-basic-principles-of-oop
3. https://www.linkedin.com/pulse/oops-java-part-3-akshay-pratap-singh-do05c